Hashing a indirizzamento aperto

Materia	Algoritmi
Anno	Secondo Anno
Data	@November 5, 2024
Studiato	Done

Un metodo di hashing che evita le collisioni è il metodo di hashing a **indirizzamento aperto**.

Sia S l'insieme da rappresentare, e sia $h(k)$ la funzione di hashing. Se k è una chiave tale che si verifica una collisione, si cerca uno spazio libero per inserire k. Ad esempio, se $h(k_1) = h(k_2)$, allora possiamo cambiare il valore di $h(k_2)$.

Bisogna definire la funzione di hashing in modo tale che **vi siano più tentativi**. Bisogna creare una funzione che associ una chiave ad una **serie di posizioni**; tale serie coincide con i **tentativi di inserimento delle chiavi**.

Naturalmente se m sono le celle disponibili, non ha senso fare più di m tentativi. Di conseguenza, i tentativi massimi sono m. La funzione può essere quindi definita nel seguente modo:

$$h(k, i) : k \in U, 0 \leq i < m$$

dove i rappresenta il **tentativo**. Ad esempio, $h(k, 0) = 3, h(k, 1) = 5$ significa che **al primo tentativo** h restituisce 3, posizione occupata, mentre **al secondo tentativo** è 5.

Generalmente possiamo quindi definire h come la funzione:

$$h : U \times \{ 0, ..., m - 1 \} \rightarrow S_m$$
$$h(k, i) \neq h(k, j) \; se \; i \neq j$$

Allora $h(k)$ è una sequenza $h(k) =< h(k, 0), h(k, 1), ..., h(k, m - 1) >$, ossia una **permutazione dell'insieme** $\{ 0, ..., m - 1 \}$. Quindi è una **sequenza di**

scansione.

Possiamo ora definire la funzione Hash-Insert:

```
hash-insert(T,k):
    # caso tabella piena
    if n = m then
        return # non inserisci l'elemento
    i = 0
    while T[h(k,i)] != null do
        i = i + 1
    T[h(k,i)] = k
    n = n + 1
```

dove n indica gli elementi in tabella, m gli elementi da rappresentare.

Nel caso pessimo, notiamo che la complessità è $O(n)$, perché in tal caso controlliamo **tutte le posizioni occupate**. Inoltre, $O(n) = O(m)$ se manca un solo elemento da inserire.

Per cercare l'elemento, invece di eseguire una ricerca lineare sulla tabella T, basta seguire la sequenza $h(k)$. Tale ricerca impiega $O(n)$ nel caso pessimo. Uno dei vantaggi nella ricerca, tuttavia, è che se durante la ricerca di un elemento troviamo una casella vuota, allora **l'elemento non è presente**, poiché se tale elemento fosse presente, allora occuperebbe tale casella vuota.

La funzione Hash-Search sarà dunque:

```
hash-search(T, k):
    i = 0
    while i < m and T[h(k,i)] != null do
        if T[h(k,i)] = k then
            return True
        i = i + 1
    return False
```

La complessità nel caso pessimo è $O(n)$.

Se invece dovessimo **eliminare un elemento** nella tabella hash, dobbiamo considerare il caso in cui la ricerca trova una casella vuota, **ma non può assumere che l'elemento non sia presente**. Possiamo, tuttavia, supporre che **ogni volta che viene eliminato un elemento**, inseriamo una etichetta D. Se tale etichetta è presente, allora la ricerca continua:

```
hash-delete(T, k):
   i = 0
   while i < m and T[h(k,i)] != null do
      if T[h(k,i)] = k then
         T[h(k,i)] = Deleted
         return True
      i = i + 1
   return False
```

E quindi dobbiamo modificare la funzione di ricerca come segue:

```
hash-search(T, k):
   i = 0
   while i < m and T[h(k,i)] != null and T[h(k,i)] != Deleted  do
      if T[h(k,i)] = k then
         return True
      i = i + 1
   return False
```

Generalmente quando eseguiamo una ricerca, dobbiamo distinguere due casi:

- la ricerca **con successo**;
- la ricerca **senza successo**.

Risulta più computazionalmente pesante la ricerca **senza successo**. Nel caso medio, il numero di ispezioni è pari a $\frac{1}{1-\alpha}$. Tale ricerca, quindi, **dipende fortemente dal fattore di carico**. In una tabella hash piena a metà, in media si eseguono 2 ispezioni. Il vantaggio è che, anche se la ricerca dipende fortemente dal fattore di carico, essa **non dipende dalla dimensione della tabella**. Anche se

$n = 10^6$, se $\alpha = \frac{9}{10}$ il numero di controlli medio è sempre 10.
Quindi la tabella ad indirizzamento aperto è più snella e impiega meno tempo nella ricerca senza successo.

Nella ricerca con successo si ha $\frac{1}{\alpha}\ln(\frac{1}{1-\alpha})$. Ad esempio, se $\alpha = 0.5$, si ha che il valore di controlli è pari a 1.3 ispezioni. Nel caso invece di $\alpha = \frac{9}{10}$, si hanno circa 2.5 ispezioni.

Costruzione di una funzione hash per indirizzamenti aperti

Ora che sappiamo come opera una funzione hash, abbiamo bisogno di una funzione in grado di restituire permutazioni che siano abbastanza diverse.

Sia P_m l'insieme di tutte le permutazioni sull'insieme $\{ 0, ..., m-1 \}$. Allora la cardinalità di tale insieme è $m!$. Sia $p \in P_m$ una permutazione. Allora la probabilità che $h(k) = p$ dev'essere la stessa per qualsiasi permutazione. Ossia si ha che:

$$P_r\{ h(k) = p \} = \frac{1}{m!} \quad \forall p \in P_m$$

Risulta complesso avere una funzione di hashing uniforme in questo caso.

Possiamo ottenere una funzione di hashing tramite una **scansione lineare**.

La funzione di hashing viene definita tramite una funzione hash ausiliaria h':

$$h' : U \to \{ 0, ..., m-1 \}$$

Allora definiamo:

$$h(k, i) = (h'(k) + i) \mod m$$

Ne segue che se $m = 10$, le nostre sequenze saranno del tipo (supponendo $h'(k) = 2$):

$$< 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2 >$$

Ci sono diverse osservazioni che vanno fatte su tale funzione. La prima tra tutte è la seguente: **le permutazioni possibili sono m perché dipendono dalla posizione**

iniziale. Tale posizione iniziale va presa da $\{ 0, ..., m-1 \}$. Questa funzione è **molto lontana dalla proprietà hashing uniforme**, perché molte permutazioni hanno probabilità 0, e le m permutazioni hanno probabilità $\frac{1}{m}$.

Un fenomeno che accade, inoltre, è il fenomeno di **agglomerazione primaria:** a **livello probabilistico la chiave da inserire ha probabilità più alta di finire in una cella vicino ad un sottoinsieme contiguo occupato della tabella T.**

Supponiamo di avere una funzione hash tramite scansione lineare. Supponiamo di avere un blocco di dimensione 2 nella tabella T e una casella vuota immediatamente successiva al blocco. La probabilità di inserire un elemento in tale casella vuota **non è $\frac{1}{m}$ ma $\frac{3}{m}$, perché verrà inserita in quella casella in tre casi:**

- il caso in cui $h'(k)$ è uguale alla casella vuota;
- il caso in cui $h'(k)$ è uguale alla casella antecedente la casella vuota, ossia la seconda casella del blocco di dimensione 2;
- il caso in cui $h'(k)$ è uguale alla prima casella del blocco di dimensione 2.

Quindi se una casella vuota è preceduta da un blocco di dimensione l, allora la probabilità che una chiave k venga inserita in tale casella vuota è $\frac{l+1}{m}$.

Quindi supponendo di avere la tabella:

$$[X, X, NULL, X, NULL, X, X, NULL, X, X]$$

Allora in posizione 2 si ha probabilità $\frac{5}{10}$, in posizione 4 si ha probabilità $\frac{2}{10}$ e infine in posizione 7 si ha probabilità $\frac{3}{10}$.

Potremmo anche definire la funzione hash come:

$$h(x, i) = (h'(x) + 2i) \mod m$$

e in tal caso dovremmo considerare non la posizione immediatamente avanti, ma **due posti avanti**. Quindi in tal caso conviene ragionare prendendo la casella e andando indietro di due posti ogni volta, e fermarsi quando si trova una casella vuota:

$$[X, X, NULL, X, NULL, X, X, NULL, X, X, NULL]$$

Allora in posizione 2 si ha probabilità $\frac{3}{11}$, in posizione 4 si ha probabilità $\frac{1}{11}$, in posizione 7 si ha la probabilità $\frac{4}{11}$, e infine in ultima posizione si ha probabilità $\frac{3}{11}$.

Alternative alla scansione lineare

Per evitare i fenomeni visti, possiamo utilizzare una **scansione quadratica**:

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \mod m$$

dove c_1, c_2 sono delle costanti scelte tali che l'**intera tabella venga scansionata dalla funzione** $h(k,i)$. Tuttavia, una volta individuati tali valori, la funzione hash si comporta meglio della scansione lineare perché non soffre di aggregazione primaria. Tuttavia, soffre di **aggregazione secondaria**, che risulta essere però molto meno grave.

Non abbiamo vantaggi rispetto alla scansione lineare per quanto riguarda le permutazioni, in quanto abbiamo sempre $\mod m$.

Hashing doppio

Siano:

$$h', h'' : U \to \{\, 0, ..., m-1 \,\}$$

dove h', h'' sono **funzioni hash diverse**.

Possiamo tali funzioni hash per realizzare una funzione:

$$h(k,i) = (h'(k) + i h''(k)) \mod m$$

Possiamo generare un numero di permutazioni diverse, perché partiamo da $h'(k)$ e poi consideriamo la seconda funzione hash $h''(k)$. Supponendo h' e h'' godano della proprietà di hashing uniforme, si ha che il numero di permutazioni totale è $m \times m = m^2$: questo perché in m modi diversi posso generare la prima posizione con $h'(k)$, ma le successive posizioni sono m. Inoltre se $k_1 \neq k_2$, anche se $h'(k_1) = h'(k_2)$ non sarà mai che $h''(k_1) = h''(k_2)$. Quindi anche nel caso peggiore, ossia che $h'(k_1) = h'(k_2)$ per ogni k_1, k_2 ho comunque m permutazioni.